



SIGNAL SERIES • DELIVER

DELIVERY, DEPLOYMENT & OPERATIONS

Releasable always, deployed by decision, watched in production.

By **Nabawy**

AUTHOR & COMPILER

Strategies

Table of Contents

03	Strategies	03	04	Mechanics	04
05	Check & Handoff	05	06	Part 1: Continuous Delivery	06
07	Part 1: Continuous Delivery · continued	07	08	6. Staging vs Production	08
09	6. Staging vs Production · continued	09	10	3. Failure Modes	10
11	3. Failure Modes · continued	11	12	4. Release Trains & Promotion Paths	12
13	4. Release Trains & Promotion Paths · continued	13	14	4. Preview Environments, Automated	14
15	Continuous Deployment	15	16	Environments & Release	16
17	Environments & Release	17	18	IaC for Environments	18
19	IaC for Environments	19	20	Part 1: Observability & Feedback	20
21	Part 1: Observability & Feedback · continued	21	22	7. Best Practices	22
23	7. Best Practices · continued	23	24	4. Recovery Drills (MTTR Practice)	24
25	4. Recovery Drills (MTTR Practice) · continued	25	26	2. Secrets Discipline	26
27	2. Secrets Discipline · continued	27	28	Rollback & Recovery	28
29	Rollback & Recovery	29	30	Related Topics	30
31	CI/CD Security	31	32	CI/CD Security	32
33	 Quick Revision — Terms Table	33	34	 Concept Questions — Distilled Answers	34
35	 Command Cheat Sheet	35	36	 Self-Quiz + Answer Key	36

▪ **How to use this book** legend

 NOTE · context  KEY · must retain  WATCH OUT · gotcha  VERIFIED · check

NOTE Read cover to cover once, then use the Contents to jump. Every chapter ends with a  EXAM box — if you can answer it cold, move on.

Strategies

Picking up from *Delivery*, where the approval gate opened — we now choose *how the new version meets real traffic*.

01 / FIVE SHAPES

Strategy	Downtime	Rollback
Recreate	Yes	Slow (redeploy)
Rolling	No	Gradual
Blue/Green	No	Instant flip-back
Canary	No	Fast drain
Feature Flags	No	Instant toggle

PICK BY BLAST RADIUS rolling by default · blue/green when no partial fleet may run bad code · canary for production proof · flags for kill-switches. Stateful data? Expand-contract migrations first — strategy alone can't save schemas.

Threads: [Approval Gates](#) · [Rollback & Recovery](#)

Mechanics

02 / HOW EACH MOVES TRAFFIC



ROLLING batch by batch, probes gate each step



BLUE/GREEN validate Green, flip router, keep Blue warm



CANARY 5 → 100% golden signals decide; halt on burn

03 / FAILURE MODES

- Rolling without probes → bad version reaches 100% "successfully".
- Blue/Green sharing mutable state → Green writes what Blue can't read; keep schemas backward-compatible.
- Canary without version labels → blended metrics lie.
- Flag sprawl → untestable combinations; every flag gets an owner + expiry.

MIGRATION SEQUENCING version every migration as ordered reversible scripts · ship app code compatible with both schemas first · back up, migrate, then ship schema-dependent code · rehearse restore on staging with production-scale data.

BRANCH BY ABSTRACTION big refactor without a long branch: wrap the target in a seam → build the replacement off the live path → flip delegation → delete the old code. Main stays releasable throughout.

Threads: [Signals & SLOs](#) · [Trunk Discipline](#)

Check & Handoff

04 / INTERVIEW CHECK

Q1 Canary vs Rolling?

Rolling replaces capacity; canary tests a hypothesis on a subset with explicit success criteria.

Q2 Why do stateful systems break strategy thinking?

Traffic shapes can't fix data: migrations must be expand-contract and backward-compatible first.

05 / HANDOFF

Strategy chosen — but canary is blind without signals. Continue in **Observability & Feedback**, whose golden signals gate every progressive rollout.

🎓 **EXAM** Deployment strategies in one breath: recreate (downtime), **rolling** (batches behind probes), **blue/green** (validate Green, flip router, instant flip-back), **canary** (5 to 100 percent on golden signals), **flags** (instant kill-switch); pick by **blast radius**, keep schemas backward-compatible with **expand-contract** first — traffic shapes cannot fix data.

▶▶ **Next: Continuous Delivery**

Part 1: Continuous Delivery

PART 1: CONTINUOUS DELIVERY

Continuous Delivery: every change that passes CI is releasable — deployment to staging is automatic, to production is a decision (approval). Continuous Deployment removes even that gate.

1. WHAT IS IT?

Picking up from **Artifact Management**, where the digest was frozen — we now move that exact artifact through staging to production.

- **Continuous Delivery:** pipeline auto-deploys to staging; production needs an approval/policy gate.
- **Continuous Deployment:** pipeline auto-deploys to production too.
- **Environment:** a runtime context — staging (prod-like rehearsal) vs production (real users).
- **Release:** a version made available; **Deployment:** installing it somewhere; **Promotion:** approving the same artifact for the next environment.

DELIVERY VS DEPLOYMENT

One gate of difference:

Continuous Delivery	Continuous Deployment	
After CI passes	Releasable, auto-staged	Shipped to production
Production deploy	A human/policy decision	Automatic
Motto	"We could ship anytime"	"We do ship every time"
Risk control	Approval gate	Progressive delivery + fast rollback

2. WHY DOES IT EXIST?

Threads: [Artifact Digest](#) · [Approval Gates](#)

Part 1: Continuous Delivery • continued

Merging is not shipping. Delivery closes the gap: staging proves the artifact in prod-like conditions, approvals control risk, and releases become routine instead of events.

3. WHERE DOES IT FIT?

```
1. merge
2. build
3. test
4. deploy to staging
5. approved for production
6. => [rejected] | Feedback -> fix
```

4. HOW IT WORKS

1. Post-merge pipeline pushes artifact; CD auto-deploys its digest to staging.
2. Automated smoke tests + (optional) manual QA run in staging.
3. Approval gate: human click, or policy gate that auto-approves when all checks are green, the deploy lands inside the allowed time window (change window — e.g. business hours, never Friday night), and the code owner has signed off.
4. CD deploys the *same digest* to production using per-environment config (env vars, secrets, scaling).
5. Post-deploy verification (health checks, error budget — the allowable failure quota; if it burns too fast, releases halt) confirms success.

5. EXAMPLE

Helm (the package manager for Kubernetes) installs the chart with the tested image tag — same digest, environment-specific values:

```
1. helm template --set image.tag=${SHA}
2. helm install
3. helm upgrade
4. helm upgrade --set image.tag=${SHA}
5. helm upgrade --set image.tag=${SHA}
6. helm upgrade --set image.tag=${SHA}
7. helm upgrade --set image.tag=${SHA}
8. helm upgrade --set image.tag=${SHA}
9. helm upgrade --set image.tag=${SHA}
10. helm upgrade --set image.tag=${SHA}
11. helm upgrade --set image.tag=${SHA}
12. helm upgrade --set image.tag=${SHA}
13. helm upgrade --set image.tag=${SHA}
14. helm upgrade --set image.tag=${SHA}
15. helm upgrade --set image.tag=${SHA}
16. helm upgrade --set image.tag=${SHA}
17. helm upgrade --set image.tag=${SHA}
18. helm upgrade --set image.tag=${SHA}
19. helm upgrade --set image.tag=${SHA}
20. helm upgrade --set image.tag=${SHA}
21. helm upgrade --set image.tag=${SHA}
22. helm upgrade --set image.tag=${SHA}
23. helm upgrade --set image.tag=${SHA}
24. helm upgrade --set image.tag=${SHA}
25. helm upgrade --set image.tag=${SHA}
26. helm upgrade --set image.tag=${SHA}
27. helm upgrade --set image.tag=${SHA}
28. helm upgrade --set image.tag=${SHA}
29. helm upgrade --set image.tag=${SHA}
30. helm upgrade --set image.tag=${SHA}
31. helm upgrade --set image.tag=${SHA}
32. helm upgrade --set image.tag=${SHA}
33. helm upgrade --set image.tag=${SHA}
34. helm upgrade --set image.tag=${SHA}
35. helm upgrade --set image.tag=${SHA}
36. helm upgrade --set image.tag=${SHA}
37. helm upgrade --set image.tag=${SHA}
38. helm upgrade --set image.tag=${SHA}
39. helm upgrade --set image.tag=${SHA}
40. helm upgrade --set image.tag=${SHA}
41. helm upgrade --set image.tag=${SHA}
42. helm upgrade --set image.tag=${SHA}
43. helm upgrade --set image.tag=${SHA}
44. helm upgrade --set image.tag=${SHA}
45. helm upgrade --set image.tag=${SHA}
46. helm upgrade --set image.tag=${SHA}
47. helm upgrade --set image.tag=${SHA}
48. helm upgrade --set image.tag=${SHA}
49. helm upgrade --set image.tag=${SHA}
50. helm upgrade --set image.tag=${SHA}
51. helm upgrade --set image.tag=${SHA}
52. helm upgrade --set image.tag=${SHA}
53. helm upgrade --set image.tag=${SHA}
54. helm upgrade --set image.tag=${SHA}
55. helm upgrade --set image.tag=${SHA}
56. helm upgrade --set image.tag=${SHA}
57. helm upgrade --set image.tag=${SHA}
58. helm upgrade --set image.tag=${SHA}
59. helm upgrade --set image.tag=${SHA}
60. helm upgrade --set image.tag=${SHA}
61. helm upgrade --set image.tag=${SHA}
62. helm upgrade --set image.tag=${SHA}
63. helm upgrade --set image.tag=${SHA}
64. helm upgrade --set image.tag=${SHA}
65. helm upgrade --set image.tag=${SHA}
66. helm upgrade --set image.tag=${SHA}
67. helm upgrade --set image.tag=${SHA}
68. helm upgrade --set image.tag=${SHA}
69. helm upgrade --set image.tag=${SHA}
70. helm upgrade --set image.tag=${SHA}
71. helm upgrade --set image.tag=${SHA}
72. helm upgrade --set image.tag=${SHA}
73. helm upgrade --set image.tag=${SHA}
74. helm upgrade --set image.tag=${SHA}
75. helm upgrade --set image.tag=${SHA}
76. helm upgrade --set image.tag=${SHA}
77. helm upgrade --set image.tag=${SHA}
78. helm upgrade --set image.tag=${SHA}
79. helm upgrade --set image.tag=${SHA}
80. helm upgrade --set image.tag=${SHA}
81. helm upgrade --set image.tag=${SHA}
82. helm upgrade --set image.tag=${SHA}
83. helm upgrade --set image.tag=${SHA}
84. helm upgrade --set image.tag=${SHA}
85. helm upgrade --set image.tag=${SHA}
86. helm upgrade --set image.tag=${SHA}
87. helm upgrade --set image.tag=${SHA}
88. helm upgrade --set image.tag=${SHA}
89. helm upgrade --set image.tag=${SHA}
90. helm upgrade --set image.tag=${SHA}
91. helm upgrade --set image.tag=${SHA}
92. helm upgrade --set image.tag=${SHA}
93. helm upgrade --set image.tag=${SHA}
94. helm upgrade --set image.tag=${SHA}
95. helm upgrade --set image.tag=${SHA}
96. helm upgrade --set image.tag=${SHA}
97. helm upgrade --set image.tag=${SHA}
98. helm upgrade --set image.tag=${SHA}
99. helm upgrade --set image.tag=${SHA}
100. helm upgrade --set image.tag=${SHA}
```

Threads: [Artifact Digest](#) • [Approval Gates](#)

6. Staging vs Production

6. STAGING VS PRODUCTION

Staging	Production	
Purpose	Rehearse, catch env issues	Serve users
Data	Sanitized/anonymized	Real
Deploys	Automatic	Gated
Scale	Smaller but representative	Full

7. FAILURE MODES

- Config drift (staging $\neq$ prod) $\rightarrow$ env-specific bugs; fix with same chart/manifests, env-only values.
- Approval bottleneck (every typo needs a manager); fix with policy gates for low-risk paths.
- Deploying a rebuild instead of the tested digest; fix by pinning digests end-to-end.

8. BEST PRACTICES

- Same artifact digest from staging to production, always.
- Separate config from code (env vars/secrets per environment).
- Small batches + feature flags (toggles switching code paths at runtime without redeploying) so approvals stay fast and safe.
- Every production deploy must be observable and reversible (see next doc).

9. COMMON MISCONCEPTIONS

- "Delivery = Deployment." Delivery keeps the decision; Deployment automates it.
- "Staging guarantees prod success." It reduces risk; traffic, data, and scale still differ — hence deployment strategies (canary, blue-green).

10. INTERVIEW / EXAM NOTES

- Delivery vs Deployment in one sentence each.
- Design an approval gate: who/what approves, on what signal.
- Why staging must mirror production's topology but never its data.

Threads: [Approval Gates](#) · [Artifact Digest](#)

6. Staging vs Production • continued

The release is live — now it must be watched. Continue in **Observability & Feedback**, where production signals close the loop back to the developer.

RELATED TOPICS

- [Artifact Management](#)
- [Observability & Feedback](#)

PART 2: CONTINUOUS DEPLOYMENT

Continuous Deployment removes the last human gate: every change passing all automated checks reaches production by itself. Freedom bought with automation rigor most teams haven't built yet.

1. WHAT IS IT?

Picking up from **Continuous Delivery**, where a human approved production — here the pipeline approves itself. Delivery asks "could we ship?"; Deployment ships.

2. PREREQUISITES (ALL NON-NEGOTIABLE)

- **Full automation:** build, test, scan, staging deploy, smoke, promotion — zero manual steps. Any human errand becomes the bottleneck that breaks the model.
- **Progressive delivery:** canary or flags on every release (see Deployment Strategies) — automation needs a small blast radius to be safe.
- **Kill criteria:** automated halt conditions (error-budget burn, p99 regression, business KPI drop) that stop and roll back without asking.
- **Observability:** version-labeled everything, SLOs with burn alerts, on-call that trusts the automation (see Observability).
- **Trunk discipline + flags:** unfinished work hides behind flags; branches live hours-to-days, never weeks — main is always shippable.

Threads: [Artifact Digest](#) • [Trunk Discipline](#)

3. Failure Modes

3. FAILURE MODES

- Automating a flaky pipeline → auto-shipping bugs at machine speed; fix gates first, automate second.
- No kill criteria → bad release reaches 100% "successfully"; every auto-promotion needs an automated halt.
- Database migrations without expand-contract → auto-deploy corrupts state; migrations must be backward-compatible across versions.
- On-call distrusts automation → humans re-add manual gates; fix with practiced, boring rollbacks that build confidence.

4. DELIVERY VS DEPLOYMENT: WHEN EACH

- **Delivery** (human gate): regulated industries, data migrations, marketing-coordinated launches, teams still building gate maturity.
- **Deployment** (no gate): SaaS with progressive delivery, strong SLOs, practiced rollback, and trunk discipline. Start with low-risk services, expand the blast radius as confidence compounds.

5. INTERVIEW NOTES

- Five prerequisites from memory, and how to verify each exists.
- Why automating a bad pipeline multiplies harm.
- Expand-contract migrations: why they're the price of auto-deploy.

RELATED TOPICS

- Continuous Delivery
- Deployment Strategies
- Observability & Feedback

Auto-shipping needs somewhere to land safely. Continue in **Environments & Release**.

🎒 **EXAM** Delivery vs Deployment in one breath: **Delivery** auto-deploys to staging and holds production behind an **approval** (human click or policy gate: green checks, change window, owner sign-off); **Deployment** automates even that gate, so it demands progressive delivery, expand-contract migrations, and the **same digest** from staging to production.

▶▶ Next: Environments & Release

Threads: [Approval Gates](#) · [Artifact Digest](#) · [Rollback & Recovery](#)

3. Failure Modes • continued

PART 3: ENVIRONMENTS & RELEASE

Environments are runtime contexts with different data, scale, and risk; release is the discipline of moving one artifact across them safely — same binary, different config, explicit promotion paths.

1. WHAT IS IT?

Picking up from **Continuous Deployment**, where automation needed safe landing zones — here is how those zones are built and governed.

2. ENVIRONMENT TOPOLOGY

Env	Purpose	Data	Deploys
Dev	Individual velocity	Fake/local	Continuous, anything
Preview (per-PR)	Review the actual change	Sanitized snapshot	Auto per PR, destroyed on merge
Staging	Prod rehearsal	Sanitized/anonymized prod-like	Auto on main
Production	Real users	Real	Gated (delivery) or automated (deployment)

Preview environments are the highest-leverage addition: reviewers click the change instead of imagining it.

3. CONFIG VS CODE

The artifact never changes between environments; everything else does via config: env vars for simple values, secret stores for credentials, config maps/files for structured settings. Inject each setting at the latest safe point — build-time injection forces per-environment artifacts and voids test-what-you-ship; deploy- or runtime injection keeps one binary everywhere. Twelve-factor discipline: a diff between staging and production behavior must be explainable by config diff alone — never by "we built it differently."

Threads: [Artifact Digest](#)

4. Release Trains & Promotion Paths

4. RELEASE TRAINS & PROMOTION PATHS

- **Release train:** scheduled departures (e.g. Tuesdays/Thursdays); whatever is green and approved rides, the rest waits for the next train. Predictable for stakeholders, calming for on-call.
- **Promotion path:** dev → preview → staging → production, each gate explicit (checks + approvals + signals). Skipping environments is the process smell that precedes incidents.
- **Hotfix lane:** a documented fast path (fewer approvals, same checks) for production fires — used rarely, reviewed always.
- **Release readiness checklist:** owners named, config externalized, environments IaC-built, approvals + monitors + data migration + back-out plan + support handoff. No checklist, no release.

5. FAILURE MODES

- Snowflake environments (hand-configured staging) → "works in staging" lies; environments are code (IaC), rebuilt identically.
- Real prod data in staging → breach waiting to happen; sanitize/anonymize with production-like shape.
- Config in code (URLs baked into images) → rebuild per env, voiding artifact immutability; externalize everything.

6. INTERVIEW NOTES

- Why preview environments beat screenshots for review quality.
- Config-vs-code: what breaks when the artifact differs per env.
- Design a hotfix lane that stays safe under pressure.

RELATED TOPICS

- Continuous Delivery
- Continuous Deployment
- Deployment Strategies

Zones built and governed — now lock down the supply chain running through them. Continue in **CI/CD Security**.

Threads: [Approval Gates](#) · [Artifact Digest](#)

4. Release Trains & Promotion Paths • continued

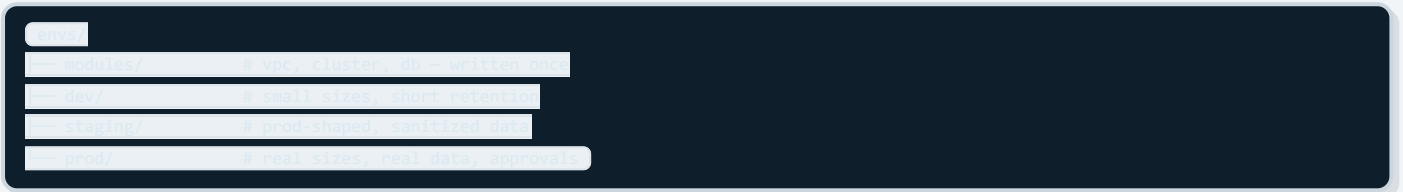
PART 4: IAC FOR ENVIRONMENTS

Environments are too important to be clicked into existence: Terraform (or equivalent) declares every environment's infrastructure so staging and production differ only by config — never by history.

1. WHAT IS IT?

Picking up from **ArgoCD & GitOps**, where app state became declarative — IaC does the same for everything underneath: VPCs, clusters, databases, DNS, IAM. Same modules, different inputs per environment.

2. STRUCTURE THAT SCALES



- **State per environment** (never shared): one blast radius per state file; remote backend with locking.
- **Modules pinned by version** — floating module sources are `:latest` with extra steps.
- **Env diff = input diff**: instance sizes, counts, secrets references. A behavioral difference not explainable by inputs is a bug.

3. IAC IN THE PIPELINE

- `terraform plan` on every PR touching `envs/` — the plan is the review.
- `apply` gated like any production deployment (approvals on prod, auto on dev).
- **Drift detection** on schedule: `plan` nightly, alert on unexpected diffs — the IaC equivalent of ArgoCD's self-heal.
- Jenkins controller itself can be code too (JCasC — see Jenkins Advanced).

Threads: [GitOps Pull Model](#) • [Approval Gates](#)

4. Preview Environments, Automated

4. PREVIEW ENVIRONMENTS, AUTOMATED

Combine the pieces: PR → pipeline runs a Terraform workspace (or ArgoCD ApplicationSet) → ephemeral env with sanitized data → destroy on merge. The highest-leverage environment pattern, fully automated, zero snowflakes.

5. FAILURE MODES

- Shared state across envs → dev `apply` deletes prod tables; separate states, separate credentials.
- Unreviewed `apply` on prod → treat IaC changes like code: PR, plan output, approval.
- Secrets in state files → state holds secrets in plaintext; encrypt backends, restrict access, prefer external secret stores.
- ClickOps drift → someone "fixes" prod in the console; drift alerts + periodic `apply` reconciliation.

6. INTERVIEW NOTES

- Why state-per-environment (blast radius).
- Plan-as-review: what makes IaC changes safe to approve.
- Where JCasC fits (controller config as code, same discipline).

RELATED TOPICS

- Environments & Release
- ArgoCD & GitOps
- Jenkins Advanced

Infrastructure declared — the library's platform arc is complete. Return to the **Topic Index**.

Threads: [GitOps Pull Model](#) · [Approval Gates](#)

Continuous Deployment

Continuous Deployment removes the last human gate: every change passing all automated checks reaches production by itself. Freedom bought with automation rigor most teams haven't built yet.

1. WHAT IS IT?

Picking up from **Continuous Delivery**, where a human approved production — here the pipeline approves itself. Delivery asks "could we ship?"; Deployment ships.

2. PREREQUISITES (ALL NON-NEGOTIABLE)

- **Full automation:** build, test, scan, staging deploy, smoke, promotion — zero manual steps. Any human errand becomes the bottleneck that breaks the model.
- **Progressive delivery:** canary or flags on every release (see Deployment Strategies) — automation needs a small blast radius to be safe.
- **Kill criteria:** automated halt conditions (error-budget burn, p99 regression, business KPI drop) that stop and roll back without asking.
- **Observability:** version-labeled everything, SLOs with burn alerts, on-call that trusts the automation (see Observability).
- **Trunk discipline + flags:** unfinished work hides behind flags; branches live hours-to-days, never weeks — main is always shippable.

3. FAILURE MODES

- Automating a flaky pipeline → auto-shipping bugs at machine speed; fix gates first, automate second.
- No kill criteria → bad release reaches 100% "successfully"; every auto-promotion needs an automated halt.
- Database migrations without expand-contract → auto-deploy corrupts state; migrations must be backward-compatible across versions.
- On-call distrusts automation → humans re-add manual gates; fix with practiced, boring rollbacks that build confidence.

4. DELIVERY VS DEPLOYMENT: WHEN EACH

- **Delivery** (human gate): regulated industries, data migrations, marketing-coordinated launches, teams still building gate maturity.
- **Deployment** (no gate): SaaS with progressive delivery, strong SLOs, practiced rollback, and trunk discipline. Start with low-risk services, expand the blast radius as confidence compounds.

5. INTERVIEW NOTES

- Five prerequisites from memory, and how to verify each exists.
- Why automating a bad pipeline multiplies harm.
- Expand-contract migrations: why they're the price of auto-deploy.

RELATED TOPICS

- Continuous Delivery
- Deployment Strategies
- Observability & Feedback

Auto-shipping needs somewhere to land safely. Continue in **Environments & Release**.

Threads: [Rollback & Recovery](#) · [Trunk Discipline](#)

Environments & Release

Environments are runtime contexts with different data, scale, and risk; release is the discipline of moving one artifact across them safely — same binary, different config, explicit promotion paths.

1. WHAT IS IT?

Picking up from **Continuous Deployment**, where automation needed safe landing zones — here is how those zones are built and governed.

2. ENVIRONMENT TOPOLOGY

Env	Purpose	Data	Deploys
Dev	Individual velocity	Fake/local	Continuous, anything
Preview (per-PR)	Review the actual change	Sanitized snapshot	Auto per PR, destroyed on merge
Staging	Prod rehearsal	Sanitized/anonymized prod-like	Auto on main
Production	Real users	Real	Gated (delivery) or automated (deployment)

Preview environments are the highest-leverage addition: reviewers click the change instead of imagining it.

3. CONFIG VS CODE

The artifact never changes between environments; everything else does via config: env vars for simple values, secret stores for credentials, config maps/files for structured settings. Inject each setting at the latest safe point — build-time injection forces per-environment artifacts and voids test-what-you-ship; deploy- or runtime injection keeps one binary everywhere. Twelve-factor discipline: a diff between staging and production behavior must be explainable by config diff alone — never by "we built it differently."

4. RELEASE TRAINS & PROMOTION PATHS

Threads: [Artifact Digest](#)

DATA RULE Staging mirrors production topology but never its data — sanitized shapes only. Real data in staging is a breach waiting for an audit.

Environments & Release • continued

- **Release train:** scheduled departures (e.g. Tuesdays/Thursdays); whatever is green and approved rides, the rest waits for the next train. Predictable for stakeholders, calming for on-call.
- **Promotion path:** dev → preview → staging → production, each gate explicit (checks + approvals + signals). Skipping environments is the process smell that precedes incidents.
- **Hotfix lane:** a documented fast path (fewer approvals, same checks) for production fires — used rarely, reviewed always.
- **Release readiness checklist:** owners named, config externalized, environments IaC-built, approvals + monitors + data migration + back-out plan + support handoff. No checklist, no release.

5. FAILURE MODES

- Snowflake environments (hand-configured staging) → "works in staging" lies; environments are code (IaC), rebuilt identically.
- Real prod data in staging → breach waiting to happen; sanitize/anonymize with production-like shape.
- Config in code (URLs baked into images) → rebuild per env, voiding artifact immutability; externalize everything.

6. INTERVIEW NOTES

- Why preview environments beat screenshots for review quality.
- Config-vs-code: what breaks when the artifact differs per env.
- Design a hotfix lane that stays safe under pressure.

RELATED TOPICS

- Continuous Delivery
- Continuous Deployment
- Deployment Strategies

Zones built and governed — now lock down the supply chain running through them. Continue in **CI/CD Security**.

Threads: [Approval Gates](#) • [Artifact Digest](#)

IaC for Environments

Environments are too important to be clicked into existence: Terraform (or equivalent) declares every environment's infrastructure so staging and production differ only by config — never by history.

1. WHAT IS IT?

Picking up from **ArgoCD & GitOps**, where app state became declarative — IaC does the same for everything underneath: VPCs, clusters, databases, DNS, IAM. Same modules, different inputs per environment.

2. STRUCTURE THAT SCALES

```
envs/  
-- modules/      # vpc, cluster, db — written once  
-- dev/          # small sizes, short retention  
-- staging/       # prod-shaped, sanitized data  
-- prod/         # real sizes, real data, approvals
```

- **State per environment** (never shared): one blast radius per state file; remote backend with locking.
- **Modules pinned by version** — floating module sources are `:latest` with extra steps.
- **Env diff = input diff**: instance sizes, counts, secrets references. A behavioral difference not explainable by inputs is a bug.

3. IAC IN THE PIPELINE

- `terraform plan` on every PR touching `envs/` — the plan is the review.
- `apply` gated like any production deployment (approvals on prod, auto on dev).
- **Drift detection** on schedule: `plan` nightly, alert on unexpected diffs — the IaC equivalent of ArgoCD's self-heal.
- Jenkins controller itself can be code too (JCasC — see Jenkins Advanced).

4. PREVIEW ENVIRONMENTS, AUTOMATED

Threads: [GitOps Pull Model](#) · [Approval Gates](#)

IaC for Environments • continued

Combine the pieces: PR → pipeline runs a Terraform workspace (or ArgoCD ApplicationSet) → ephemeral env with sanitized data → destroy on merge. The highest-leverage environment pattern, fully automated, zero snowflakes.

5. FAILURE MODES

- Shared state across envs → dev `apply` deletes prod tables; separate states, separate credentials.
- Unreviewed `apply` on prod → treat IaC changes like code: PR, plan output, approval.
- Secrets in state files → state holds secrets in plaintext; encrypt backends, restrict access, prefer external secret stores.
- ClickOps drift → someone "fixes" prod in the console; drift alerts + periodic `apply` reconciliation.

6. INTERVIEW NOTES

- Why state-per-environment (blast radius).
- Plan-as-review: what makes IaC changes safe to approve.
- Where JCasC fits (controller config as code, same discipline).

RELATED TOPICS

- Environments & Release
- ArgoCD & GitOps
- Jenkins Advanced

Infrastructure declared — the library's platform arc is complete. Return to the **Topic Index**.

🎓 **EXAM** Environments & IaC in one breath: dev to **preview** (per PR, destroyed on merge) to staging to production, each **IaC-built** from versioned modules with separate state and env-only inputs; **config injects at deploy or runtime**, never baked in; **terraform plan** is the PR review, drift is detected nightly — env diff must equal input diff.

➡ Next: Observability & Feedback

Threads: [GitOps Pull Model](#) • [Approval Gates](#)

Part 1: Observability & Feedback

PART 1: OBSERVABILITY & FEEDBACK

Observability (logs, metrics, traces) reveals what a deployment actually did; feedback routes that signal back to the developer so the loop — commit → production → learning — closes.

1. WHAT IS IT?

Picking up from **Continuous Delivery**, where the digest reached production — we now watch what it does there and feed the lessons back.

- **Observability:** logs (events), metrics (numbers over time), traces (request paths) answering "what is production doing?"
- **Feedback:** every signal returned to the author: PR check results, staging smoke verdicts, prod alerts, DORA trends (DORA = DevOps Research & Assessment; its four delivery signals are listed in Section 7).

2. WHY DOES IT EXIST?

Without feedback, deployment is a cliff — code leaves the pipeline and nobody learns. The loop `deploy → observe → learn → fix` is what makes CI/CD self-improving instead of merely automated.

3. WHERE DOES IT FIT?

```
Flowchart: M
P[Production] --> O[Observability<br/>logs · metrics · traces]
O --> F[Feedback]
F --> D[Developer]
D --> G[Git → next fix]
```

Threads: [Signals & SLOs](#)

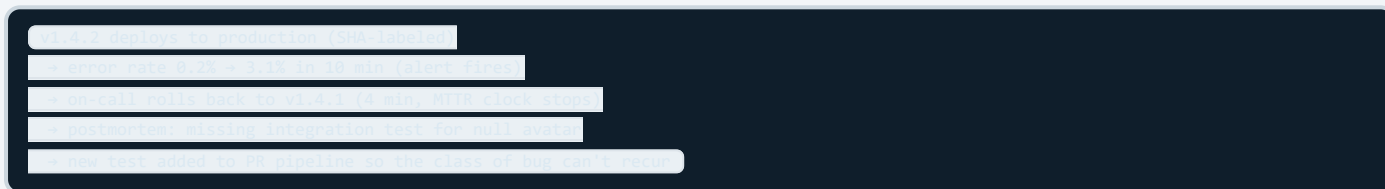
Part 1: Observability & Feedback • continued

Signal	Consumer	Action
PR checks	Developer	Fix before merge
Staging smoke	Team	Block promotion
Error-rate/latency alert	On-call	Roll back or forward-fix
Deploy frequency, MTTR (mean time to recovery) trends	Leads	Improve pipeline

4. HOW IT WORKS

1. Every deploy is labeled (version/SHA) so logs/metrics correlate with the change.
2. Dashboards track golden signals — the four user-visible health metrics: latency, traffic, errors, saturation — plus business KPIs.
3. Alerts fire on symptom (error budget burn — failures consuming the allowed quota too fast; p99 spike — the slowest 1% of requests degrading), not on every deploy.
4. Incident → revert or forward-fix → postmortem feeds back into pipeline gates (new test, new scan, tighter approval).

5. EXAMPLE



6. FAILURE MODES

- Alert fatigue (noisy thresholds) → alerts ignored; fix with SLO-based alerts (SLO = Service Level Objective, e.g. 99.9% successful requests; alert when the error budget burns, not on every blip).
- Unlabeled deploys (can't tell which version broke prod); fix with version annotations everywhere.
- Postmortems without pipeline changes → repeat incidents; every action item should land as a test, check, or runbook.

SLI / SLO / ALERTING

- **SLI** (indicator): what you measure — success ratio, p99 latency. **SLO** (objective): the promise — 99.9% success over 30 days. **Error budget**: 100% – SLO; the failures you're allowed.
- Alert on **budget burn rate**, not raw errors: fast burn (2% budget/hour) pages a human; slow burn (week-long drift) files a ticket. Every alert names its runbook.
- Keep SLOs few (3–5 per service); each SLO without an owner and a consequence is decoration.

Threads: [Signals & SLOs](#) • [Rollback & Recovery](#)

7. Best Practices

7. BEST PRACTICES

- Standardize structured logs (machine-parseable `key=value` lines, not free text) + trace IDs (one ID following a single request across every service it touches) + version labels from day one.
- Track the four DORA signals: deploy frequency, lead time, change-fail rate, MTTR.
- Make feedback fast: PR results in minutes, prod anomalies in minutes, trends weekly.

DORA METRICS (WHAT GOOD LOOKS LIKE)

Signal	Elite hint	Moves it
Deploy frequency	On demand, multiple/day	Small batches, trunk discipline, fast PR pipeline
Lead time (commit→prod)	< 1 day	Same, plus approval automation
Change-fail rate	< 15%	Better PR gates, canary, flags
MTTR	< 1 hour	Version labels, practiced rollback, clear on-call

Measure monthly, review with leads, improve one signal at a time. DORA classifies; it doesn't fix — the pipeline changes in Section 4 do.

8. INTERVIEW / EXAM NOTES

- Logs vs metrics vs traces, one use each.
- Design the rollback decision: which signal, who decides, how fast.
- How a production incident improves the CI pipeline (concrete example above).

The loop is closed conceptually — now see it implemented. Continue in the **Jenkins Domain**, where these generic concepts become controllers, agents, and Jenkinsfiles.

RELATED TOPICS

- CI/CD Overview
- Continuous Delivery
- CI/CD Pipelines

 **EXAM** Observability in one breath: every deploy ships **SHA-labeled** so logs, metrics and traces correlate to the change; watch the **golden signals** (latency, traffic, errors, saturation), alert on **error-budget burn** not blips, track **DORA** — and every incident action item lands as a pipeline gate so the class of bug cannot recur.

Next: Rollback & Recovery

Threads: [Signals & SLOs](#) · [Approval Gates](#) · [Rollback & Recovery](#)

7. Best Practices • continued

PART 2: ROLLBACK & RECOVERY

Rollback is a feature, not an apology: a practiced, boring path from bad production back to good — decided by signals, executed in minutes, reviewed afterward.

1. WHAT IS IT?

Picking up from **CI/CD Security**, where defenses were hardened — defenses still fail, so recovery must be engineered with the same rigor as release.

2. ROLLBACK PER STRATEGY

Strategy	Rollback move	Time
Rolling	Roll back batch by batch (or rollout undo)	Minutes
Blue/Green	Flip router back to Blue	Seconds
Canary	Drain canary, 100% to stable	Seconds–minutes
Feature flag	Toggle off	Instant
Recreate	Redeploy previous digest	Slowest — avoid where recovery matters

Rollback deploys a **previous known-good digest** (Artifact Management) — never a fresh build. Fresh builds during incidents add untested variables to a fire.

3. ROLLBACK VS FORWARD-FIX DECISION

- **Rollback when:** cause unknown, blast radius growing, error budget burning fast, or the fix is uncertain. Default for Sev1.
- **Forward-fix when:** cause known, fix tiny and verified, rollback itself risky (migrations already applied — data can't "roll back," only migrate forward).
- **Decider:** on-call with pre-agreed thresholds, not a committee. Decide in minutes; review in the postmortem.

Threads: [Rollback & Recovery](#) • [Artifact Digest](#)

4. Recovery Drills (MTTR Practice)

4. RECOVERY DRILLS (MTTR PRACTICE)

- Game days: inject failure in staging (kill pods, expire certs, break the pipeline), time the recovery, fix the gaps found.
- Rollback rehearsal: each service rolls back on staging quarterly until MTTR is measured, not guessed.
- Jenkins/server recovery: restore `JENKINS_HOME` backup to fresh infra, replay smoke pipeline (see Jenkins Advanced).

5. POSTMORTEMS THAT PREVENT RECURRENCE

Blameless, owner-assigned, pipeline-linked: every action item becomes a test, check, alert, or runbook — or it will recur. Track repeat-incident rate; a postmortem without a merged pipeline change is a diary entry.

6. DATA MIGRATION PRACTICE

Data outlives code and can't be discarded on deploy — migrations move forward, never back:

- Version every migration: ordered, reversible scripts run by a migration runner (not hand-applied SQL); each deploy maps to exact schema versions.
- Expand-contract sequencing: add (nullable) → migrate data → enforce; ship app code compatible with both schemas before migrating.
- Risky upgrades: deploy dual-compatible app on the old schema, observe, back up, migrate, then ship schema-dependent code.
- Verify restore, not just backup: rehearse production migration and rollback paths on staging with production-scale data.
- Test data: construct minimal data per test; never depend on full production dumps (slow, privacy-risky, unreproducible).

7. FAILURE MODES

- No practiced rollback → improvisation during incidents; rehearse until boring.
- Rolling forward into a migration → data corruption; expand-contract migrations make both directions safe.
- Rollback untested because "we never need it" → the one time it's needed, it doesn't work. Test the path you pray you never take.

Threads: [Rollback & Recovery](#)

4. Recovery Drills (MTTR Practice) • continued

8. INTERVIEW NOTES

- Rollback vs forward-fix: decision criteria in 60 seconds.
- Why rollback uses old digests, never fresh builds.
- Design a game day for a canary-deployed service.
- Rollback vs forward-fix after a migration already applied.

RELATED TOPICS

- Deployment Strategies
- Artifact Management
- Observability & Feedback
- Jenkins Advanced

Recovery engineered — the library's concept arc is complete. Return to the **Topic Index** for labs and reference.

PART 3: CI/CD SECURITY

The pipeline is production infrastructure with production secrets and the power to ship code — attack it like one: least privilege, signed artifacts, verified supply chain.

1. WHAT IS IT?

Picking up from **Environments & Release**, whose zones carry real credentials — the pipeline itself is the highest-value target in the estate: it holds deploy keys, signs releases, and runs attacker-influenced PR code.

Threads: [Rollback & Recovery](#) • [Artifact Digest](#)

2. Secrets Discipline

2. SECRETS DISCIPLINE

- Store in managers (Vault, cloud KMS, Jenkins credentials store, GitHub environments) — never in repos, images, logs, or chat.
- Scope narrowly (per-env, per-job), rotate on schedule and on any leak, audit every access.
- PR builds from forks get zero production secrets — credential-free sandboxes only (see Jenkins Agents trust boundary).

3. SIGNING, SBOM & PROVENANCE

Build once, then: sign the digest (cosign), generate the SBOM (Syft/Trivy), attest provenance (SLSA: who built, from which commit, in which pipeline). Deploy policies admit only signed, attested images — see Artifact Management. When the next CVE lands, query SBOMs to find affected digests in minutes.

4. LEAST PRIVILEGE EVERYWHERE

- CI tokens: per-repo, minimal scopes, short TTL (OIDC federation beats long-lived keys — workloads mint credentials, nobody stores them).
- Pipeline permissions: `GITHUB_TOKEN` minimal scopes; Jenkins agents can't read other jobs' secrets (agent→controller deny).
- Humans: matrix authorization, SSO + MFA, no shared admin accounts, no standing prod access without review.

5. SUPPLY-CHAIN GATES

Pin everything: action SHAs (not `@v4` tags), base image digests (not `:latest`), plugin versions, lockfiles. Verify checksums on download. Each PR that bumps a dependency re-runs the full gate — Dependabot PRs are untrusted code until proven otherwise.

6. GOVERNANCE & CHANGE EVIDENCE

Threads: [Supply-Chain Security](#) · [Artifact Digest](#) · [Controllers & Agents](#)

2. Secrets Discipline • continued

Regulated or not, every team answers "who shipped what, approved by whom, proven how" — the pipeline is the evidence:

- Maturity ladder: manual releases → versioned + automated → gated + attested → measured + continuously improved. Assess per area, fix the most painful gap first.
- Pipeline as audit trail: every production change maps to a commit, a built digest, stage results, approver identity, and target environment — automation beats paper because scripts must actually work to pass.
- Change control that scales: ticketed approvals for production, access-controlled environments, remediation plan + success criteria on every privileged change; low-risk paths auto-approve via policy gates.
- Risk in one line: severity $\approx$ impact $\times$ likelihood; compare against mitigation cost before adding ceremony.
- Value-stream mapping: when delivery feels slow, map commit-to-production steps with wait times — fix the longest wait first, not the loudest complaint.

7. FAILURE MODES

- Secret in git history → rotate immediately; history rewrites don't unsee it (it's cloned everywhere).
- Floating tags (`:latest` , `@v4`) → silent supply-chain swaps; pin digests/SHAs.
- Over-permissioned CI token → one compromised workflow owns the org; scope to the repo, expire fast.
- Unsigned images admitted → provenance theater; enforce admission, don't just generate attestations.

8. INTERVIEW NOTES

- OIDC federation vs long-lived keys for CI auth.
- Design fork-PR isolation for a public repo with deploy secrets.
- SBOM in incident response: CVE lands at 2am, what do you query first?
- Prove a production deploy was authorized: which five pipeline records do you pull?

RELATED TOPICS

- Artifact Management
- Jenkins Security
- GitHub Actions

Locked down — now plan for when things break anyway. Continue in **Rollback & Recovery**.

Threads: [Artifact Digest](#) • [Supply-Chain Security](#) • [Approval Gates](#)

Rollback & Recovery

Rollback is a feature, not an apology: a practiced, boring path from bad production back to good — decided by signals, executed in minutes, reviewed afterward.

1. WHAT IS IT?

Picking up from **CI/CD Security**, where defenses were hardened — defenses still fail, so recovery must be engineered with the same rigor as release.

2. ROLLBACK PER STRATEGY

Strategy	Rollback move	Time
Rolling	Roll back batch by batch (or rollout undo)	Minutes
Blue/Green	Flip router back to Blue	Seconds
Canary	Drain canary, 100% to stable	Seconds–minutes
Feature flag	Toggle off	Instant
Recreate	Redeploy previous digest	Slowest — avoid where recovery matters

Rollback deploys a **previous known-good digest** (Artifact Management) — never a fresh build. Fresh builds during incidents add untested variables to a fire.

3. ROLLBACK VS FORWARD-FIX DECISION

- **Rollback when:** cause unknown, blast radius growing, error budget burning fast, or the fix is uncertain. Default for Sev1.
- **Forward-fix when:** cause known, fix tiny and verified, rollback itself risky (migrations already applied — data can't "roll back," only migrate forward).
- **Decider:** on-call with pre-agreed thresholds, not a committee. Decide in minutes; review in the postmortem.

4. RECOVERY DRILLS (MTTR PRACTICE)

- Game days: inject failure in staging (kill pods, expire certs, break the pipeline), time the recovery, fix the gaps found.
- Rollback rehearsal: each service rolls back on staging quarterly until MTTR is measured, not guessed.
- Jenkins/server recovery: restore `JENKINS_HOME` backup to fresh infra, replay smoke pipeline (see Jenkins Advanced).

Threads: [Rollback & Recovery](#) · [Artifact Digest](#)

Rollback & Recovery • continued

5. POSTMORTEMS THAT PREVENT RECURRENCE

Blameless, owner-assigned, pipeline-linked: every action item becomes a test, check, alert, or runbook — or it will recur. Track repeat-incident rate; a postmortem without a merged pipeline change is a diary entry.

6. DATA MIGRATION PRACTICE

Data outlives code and can't be discarded on deploy — migrations move forward, never back:

- Version every migration: ordered, reversible scripts run by a migration runner (not hand-applied SQL); each deploy maps to exact schema versions.
- Expand-contract sequencing: add (nullable) → migrate data → enforce; ship app code compatible with both schemas before migrating.
- Risky upgrades: deploy dual-compatible app on the old schema, observe, back up, migrate, then ship schema-dependent code.
- Verify restore, not just backup: rehearse production migration and rollback paths on staging with production-scale data.
- Test data: construct minimal data per test; never depend on full production dumps (slow, privacy-risky, unreproducible).

7. FAILURE MODES

- No practiced rollback → improvisation during incidents; rehearse until boring.
- Rolling forward into a migration → data corruption; expand-contract migrations make both directions safe.
- Rollback untested because "we never need it" → the one time it's needed, it doesn't work. Test the path you pray you never take.

8. INTERVIEW NOTES

- Rollback vs forward-fix: decision criteria in 60 seconds.
- Why rollback uses old digests, never fresh builds.
- Design a game day for a canary-deployed service.
- Rollback vs forward-fix after a migration already applied.

Threads: [Rollback & Recovery](#)

Related Topics

RELATED TOPICS

- [Deployment Strategies](#)
- [Artifact Management](#)
- [Observability & Feedback](#)
- [Jenkins Advanced](#)

Recovery engineered — the library's concept arc is complete. Return to the **Topic Index** for labs and reference.

CI/CD Security

The pipeline is production infrastructure with production secrets and the power to ship code — attack it like one: least privilege, signed artifacts, verified supply chain.

1. WHAT IS IT?

Picking up from **Environments & Release**, whose zones carry real credentials — the pipeline itself is the highest-value target in the estate: it holds deploy keys, signs releases, and runs attacker-influenced PR code.

2. SECRETS DISCIPLINE

- Store in managers (Vault, cloud KMS, Jenkins credentials store, GitHub environments) — never in repos, images, logs, or chat.
- Scope narrowly (per-env, per-job), rotate on schedule and on any leak, audit every access.
- PR builds from forks get zero production secrets — credential-free sandboxes only (see Jenkins Agents trust boundary).

3. SIGNING, SBOM & PROVENANCE

Build once, then: sign the digest (cosign), generate the SBOM (Syft/Trivy), attest provenance (SLSA: who built, from which commit, in which pipeline). Deploy policies admit only signed, attested images — see Artifact Management. When the next CVE lands, query SBOMs to find affected digests in minutes.

4. LEAST PRIVILEGE EVERYWHERE

- CI tokens: per-repo, minimal scopes, short TTL (OIDC federation beats long-lived keys — workloads mint credentials, nobody stores them).
- Pipeline permissions: `GITHUB_TOKEN` minimal scopes; Jenkins agents can't read other jobs' secrets (agent→controller deny).
- Humans: matrix authorization, SSO + MFA, no shared admin accounts, no standing prod access without review.

5. SUPPLY-CHAIN GATES

Pin everything: action SHAs (not `@v4` tags), base image digests (not `:latest`), plugin versions, lockfiles. Verify checksums on download. Each PR that bumps a dependency re-runs the full gate — Dependabot PRs are untrusted code until proven otherwise.

Threads: [Supply-Chain Security](#) · [Artifact Digest](#) · [Controllers & Agents](#)

CI/CD Security • continued

6. GOVERNANCE & CHANGE EVIDENCE

Regulated or not, every team answers "who shipped what, approved by whom, proven how" — the pipeline is the evidence:

- Maturity ladder: manual releases → versioned + automated → gated + attested → measured + continuously improved. Assess per area, fix the most painful gap first.
- Pipeline as audit trail: every production change maps to a commit, a built digest, stage results, approver identity, and target environment — automation beats paper because scripts must actually work to pass.
- Change control that scales: ticketed approvals for production, access-controlled environments, remediation plan + success criteria on every privileged change; low-risk paths auto-approve via policy gates.
- Risk in one line: severity $\approx$ impact $\times$ likelihood; compare against mitigation cost before adding ceremony.
- Value-stream mapping: when delivery feels slow, map commit-to-production steps with wait times — fix the longest wait first, not the loudest complaint.

7. FAILURE MODES

- Secret in git history → rotate immediately; history rewrites don't unsee it (it's cloned everywhere).
- Floating tags (`:latest` , `@v4`) → silent supply-chain swaps; pin digests/SHAs.
- Over-permissioned CI token → one compromised workflow owns the org; scope to the repo, expire fast.
- Unsigned images admitted → provenance theater; enforce admission, don't just generate attestations.

8. INTERVIEW NOTES

- OIDC federation vs long-lived keys for CI auth.
- Design fork-PR isolation for a public repo with deploy secrets.
- SBOM in incident response: CVE lands at 2am, what do you query first?
- Prove a production deploy was authorized: which five pipeline records do you pull?

RELATED TOPICS

- Artifact Management
- Jenkins Security
- GitHub Actions

Locked down — now plan for when things break anyway. Continue in **Rollback & Recovery**.

🎓 **EXAM** Recovery & Security in one breath: roll back a previous **known-good digest** (flip, drain, toggle — never a fresh build), forward-fix only when the cause is known and migrations already applied; rehearse until boring with **game days** and blameless postmortems — while secrets stay in **managers**, digests stay **signed with SBOMs**, and every dependency stays **pinned**.

Threads: [Artifact Digest](#) • [Rollback & Recovery](#) • [Supply-Chain Security](#)



Quick Revision — Terms Table

Recreate	Stop-all then deploy; downtime, slowest rollback.
Rolling	Batch-by-batch replacement gated by probes.
Blue/Green	Validate Green, flip router, keep Blue warm; instant flip-back.
Canary	Subset rollout 5 to 100 percent decided by golden signals.
Feature flag	Runtime toggle switching code paths; instant kill-switch.
Continuous Delivery	Every change releasable; production is a decision.
Continuous Deployment	Every passing change ships automatically.
Approval gate	Human click or policy gate releasing production.
Change window	Allowed deploy hours; never Friday night.
Staging	Prod-like rehearsal; automatic deploys, sanitized data.

Promotion path	dev to preview to staging to production, each gate explicit.
Release train	Scheduled departures; green and approved rides.
Hotfix lane	Documented fast path for fires; same checks, fewer approvals.
Preview environment	Ephemeral per-PR env; destroyed on merge.
Config vs code	Artifact constant; env vars and secrets inject per environment.
Drift detection	Nightly plan surfacing unexpected infra diffs.
Golden signals	Latency, traffic, errors, saturation.
SLI / SLO	Measured indicator vs promised objective (99.9 percent).
Error budget	Allowed failures; burn rate pages, blips do not.
MTTR	Mean time to recovery; measured by drills, not guessed.

KEY IDEA Cover the right column and define each term aloud. Any stumble goes back to its chapter.

Threads: [Signals & SLOs](#) · [Approval Gates](#) · [Artifact Digest](#)



Concept Questions — Distilled Answers

Q1 Delivery vs Deployment in one line each?

Delivery: releasable behind a decision. Deployment: shipped automatically.

Q2 Canary vs Rolling?

Rolling replaces capacity; canary tests a hypothesis on a subset with success criteria.

Q3 Why the same digest from staging to production?

A rebuild voids verification — you would ship what you never tested.

Q4 Why do stateful systems break strategy thinking?

Traffic shapes cannot fix data; expand-contract migrations come first.

Q5 Rollback vs forward-fix?

Roll back by default on unknown cause or fast burn; forward-fix only for known tiny fixes over applied migrations.

Q6 SLI vs SLO vs error budget?

Indicator measured, objective promised, budget allowed — alert on burn rate.

Q7 Why do preview environments beat screenshots?

Reviewers click the actual change instead of imagining it.

Q8 ★ A deploy spikes errors at night. Walk me through it.

Halt the burn via rollback of the known-good digest, stop the MTTR clock, then postmortem the class of bug into a pipeline gate.

Threads: [Rollback & Recovery](#) · [Signals & SLOs](#) · [Approval Gates](#)

Command Cheat Sheet

Release

<code>helm upgrade app ./chart --set image.tag=\$SHA</code>	same digest, env-specific values
<code>helm upgrade app ./chart --set image.tag=\$SHA (production)</code>	same command behind approval gate

laC

<code>terraform plan (per PR on envs/)</code>	the plan is the review
<code>terraform apply (gated on prod)</code>	deploys infra like code
<code>plan nightly</code>	drift detection while you sleep

Recover

<code>rollout undo (rolling)</code>	batch-by-batch retreat
<code>flip router to Blue</code>	instant blue/green rollback
<code>toggle flag off</code>	instant kill-switch

```

deploy-staging: # auto on main
  script: helm upgrade app ./chart --set image.tag=$SHA
deploy-production: # manual approval gate
  needs: [deploy-staging, smoke-tests] when: manual
  
```

NOTE Release order: green checks, sane window, signed digest — then watch the golden signals. Never force a red gate.

Threads: [Approval Gates](#)



Self-Quiz + Answer Key

Q1 Which strategy rolls back instantly?

Q2 What separates Delivery from Deployment?

Q3 Preview environments live how long?

Q4 State the config-vs-code rule.

Q5 Alert on what, not on what?

Q6 Default on unknown cause with fast burn?

Q7 Where do secrets live?

Q8 What does a postmortem produce?

▪ Answer key

1. Blue/green flip-back or a feature-flag toggle.
2. One gate: a human or policy decision before production.
3. Per PR; destroyed on merge.
4. Same artifact everywhere; env diff must equal config diff.
5. On error-budget burn rate, not on every blip.
6. Roll back the known-good digest; Sev1 defaults to rollback.
7. In managers per env and job — never repos, images, logs, or chat.
8. A landed pipeline change: test, check, alert, or runbook.

FINAL CHECK Eight for eight from memory, twice in a row — the delivery arc is yours. The concept arc is complete.

Threads: [Artifact Digest](#) • [Approval Gates](#) • [Rollback & Recovery](#)